Relatório Técnico da Colônia Aurora

Controle de energia da colônia · Fase 3 · Atividade do Capítulo 1

Integrantes: Ronei Davi Lach, Gabriel Hampel Meireles, Roberto Kenji Iuvata, José Roberto Zendron e David Silva

Repositório da equipe: https://github.com/rdlach/fiap-fase3-atividade-capitulo1-colonia-inteligente

1. Introdução

O programa organiza os dados da colônia Aurora Siger, estima a geração solar e eólica e compara essa energia com o consumo dos equipamentos. Quando as regras pedem economia ou emergência, ele desliga os equipamentos menos importantes e mantém os de prioridade 1 ligados.

Este relatório descreve a versão atual do código da equipe. O programa executa um ciclo no terminal, com dados fixos de exemplo. O cenário começa com as seguintes informações:

Dado	Valor inicial	Uso no programa
Reserva da bateria	40%	Indica se a reserva está baixa ou crítica
Consumo dos equipamentos	70 kWh	Soma dos equipamentos ligados para uma hora
Temperatura externa	-63 °C	Verifica a condição de frio extremo
Vento previsto	11 m/s	Entrada da previsão eólica
Irradiação prevista	450 W/m²	Entrada da previsão solar
Tempestade de poeira	Não	Se houver poeira, reduz a previsão solar

A reserva está em **porcentagem**. A geração e o consumo estão em **kWh para uma hora**. O código não informa a capacidade da bateria em kWh e não calcula sua carga final. Por isso, não podemos subtrair um déficit em kWh diretamente dos 40% de reserva.

2. Como os dados estão organizados

Em dados.py, o dicionário estado_atual guarda as leituras do cenário. Os históricos de vento, irradiação, energia e consumo ficam em listas. A árvore dos sistemas usa dicionários com nome, prioridade, consumo, ligado e uma lista de filhos. A Figura 1 mostra os grupos e equipamentos dessa árvore.

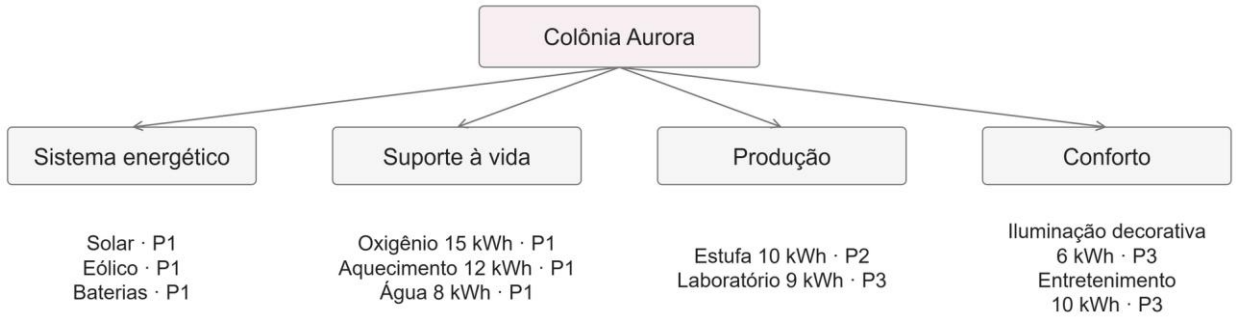


Figura 1 - Sistemas e equipamentos da árvore da colônia

P1 indica os equipamentos essenciais, P2 os de prioridade intermediária e P3 os que podem ser desligados primeiro. A função criar_indice permite buscar pelo nome, como Oxigenio. A função navegar segue um caminho, como Sistema Energetico → Solar. As fontes e as baterias aparecem na árvore com consumo zero.

3. Regras para tomar decisões

O arquivo regras.py verifica seis condições. As letras abaixo são as mesmas usadas na saída do programa. Cada condição pode ser verdadeira ou falsa.

Letra	Condição	Quando é verdadeira
A	Reserva baixa	Reserva abaixo de 50%
B	Consumo alto	Consumo acima de 60 kWh
C	Reserva crítica	Reserva abaixo de 20%
D	Frio extremo	Temperatura abaixo de -80 °C
E	Déficit previsto	Geração prevista menor que o consumo de referência
F	Tempestade de poeira	O estado informa tempestade de poeira

A emergência é verificada antes da economia. Assim, se uma situação atender às duas regras, vale a ação de emergência. As expressões principais são:

```
EMERGENCIA = C OR (D AND A)
ECONOMIA = NOT EMERGENCIA AND A AND B
```

Modo	Condição	O que o código faz
EMERGENCIA	Reserva < 20%, ou temperatura < -80 °C com reserva < 50%	Desliga os equipamentos de prioridades 2 e 3
ECONOMIA	Fora da emergência, reserva < 50% e consumo > 60 kWh	Desliga somente os equipamentos de prioridade 3
ATENCAO	Déficit previsto e reserva de pelo menos 50%	Emite aviso preventivo; não desliga equipamentos automaticamente
NORMAL	Nenhuma condição anterior muda o modo	Mantém os equipamentos ligados; ainda pode mostrar avisos

3.1 Avisos que acompanham o modo

Sempre que A é verdadeira, aparece o alerta para reduzir consumo. Esse aviso pode acompanhar economia ou emergência. Também pode aparecer com modo NORMAL: por exemplo, reserva de 45% e consumo de 50 kWh geram o alerta, mas não atendem à regra de corte automático.

Se E for verdadeira e a reserva não estiver baixa, o programa avisa que a previsão não cobre o consumo e muda NORMAL para ATENCAO. Se houver poeira, acrescenta a orientação de priorizar a energia eólica e as baterias. Esse aviso de clima pode aparecer junto com os outros modos.

3.2 Resultado do cenário inicial

Com reserva de 40%, consumo de 70 kWh e temperatura de -63 °C, temos A e B verdadeiras, mas C e D falsas. O modo escolhido é **ECONOMIA**. A geração prevista também é menor que 70 kWh, mas o aviso preventivo de ATENCAO só aparece quando a reserva é de pelo menos 50%. Os equipamentos de prioridade 1 não são desligados pelas regras.

4. Previsão da geração de energia

O arquivo previsao.py calcula uma reta por mínimos quadrados. A eólica usa o vento como entrada; a solar usa a irradiação. O cenário completo usa **cinco pares em cada histórico**, mostrados abaixo.

Histórico	1	2	3	4	5
Vento (m/s)	8	10	12	6	14
Energia eólica (kWh)	20	26	29	14	36
Irradiação (W/m²)	200	300	400	500	600
Energia solar (kWh)	11	14	21	24	31

4.1 Retas e resultados do cenário

A função ajustar_reta encontra b0 e b1 da fórmula y = b0 + b1 × x. A função prever aplica essa reta ao vento ou à irradiação esperados. Para os dados atuais, as contas são:

```
Eolica: y = -1,50 + 2,65 * vento
Vento 11 m/s: y = 27,65 kWh
Solar: y = 0,20 + 0,050 * irradiacao
Irradiacao 450 W/m2: y = 22,70 kWh
```

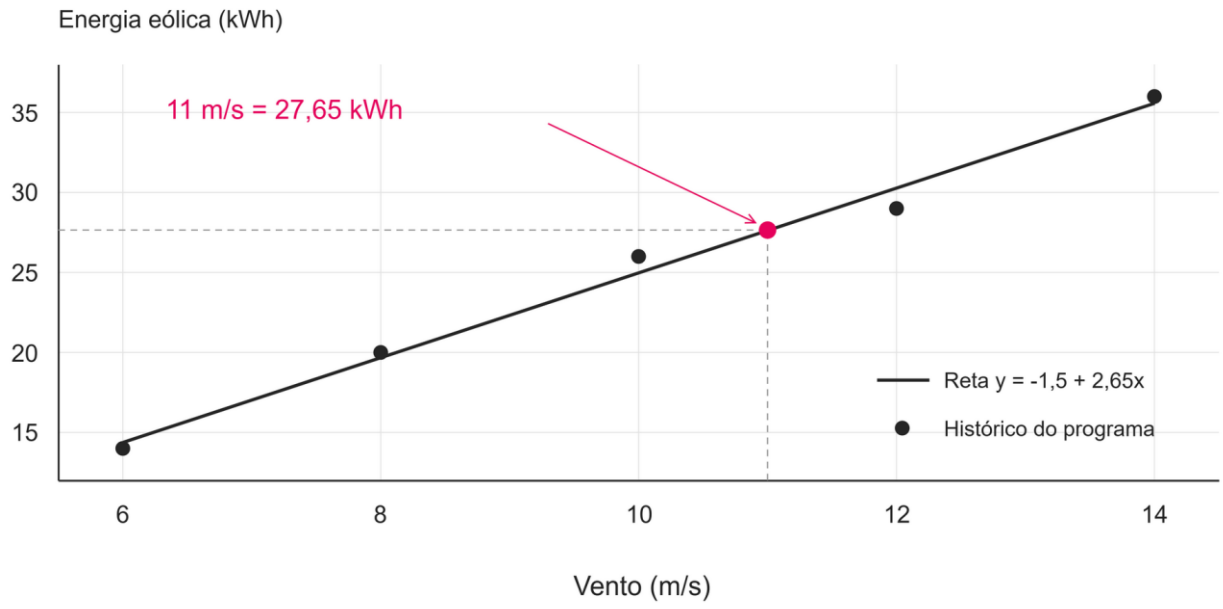


Figura 2 - Regressão eólica do cenário atual e previsão para 11 m/s

A geração total prevista é **27,65 + 22,70 = 50,35 kWh**. O erro absoluto médio da eólica no próprio histórico de ajuste é 0,68 kWh. Esse valor mostra o ajuste aos dados usados, não o erro em medições futuras.

4.2 Consumo histórico e exemplo do enunciado

A lista de consumo é [55, 60, 58, 65, 70]. A função tendencia estima **72,1 kWh** para a próxima hora, com aumento de 3,5 kWh por hora na reta. Essa estimativa é apenas informativa: o saldo usa os 70 kWh dos equipamentos ligados e, depois dos cortes, os 45 kWh restantes.

Os **27,5 kWh** continuam corretos para o exemplo separado do enunciado, com vento [8, 10, 12] e energia [20, 25, 30]. Esse exemplo aparece no fim da execução e não deve ser confundido com os 27,65 kWh calculados com os cinco pares do cenário completo.

5. Consumo antes e depois da economia

O programa soma apenas o consumo dos equipamentos ligados. No cenário inicial, eles pedem 70 kWh para a próxima hora, se continuarem ligados. A geração prevista é de 50,35 kWh, então o saldo inicial é **50,35 - 70 = -19,65 kWh**. A geração cobre 71,9% desse consumo.

No modo economia, desligar_por_prioridade desliga a prioridade 3. O suporte à vida e a estufa continuam ligados. A tabela mostra os valores que estão em dados.py e o resultado desse corte.

Equipamento	Prioridade	Antes (kWh)	Após economia (kWh)
Oxigênio	1	15	15
Aquecimento	1	12	12
Água	1	8	8
Estufa	2	10	10
Laboratório	3	9	0
Iluminação decorativa	3	6	0
Entretenimento	3	10	0
Total	-	70	45

A economia é de **25 kWh**: 9 do laboratório, 6 da iluminação decorativa e 10 do entretenimento. O consumo passa para 45 kWh e o novo saldo é **50,35 - 45 = +5,35 kWh**. A nova análise sugere armazenar o excedente. O estado do programa também passa a registrar consumo de 45 kWh.

5.1 O que a análise informa

Geração / consumo	Saldo	Resultado
40 / 70 kWh	-30 kWh	Alerta: consumo maior que geração
80 / 30 kWh	+50 kWh	Sugestão: armazenar o excedente
40 / 40 kWh	0 kWh	Geração e consumo em equilíbrio

A reserva em porcentagem serve para detalhar os alertas: abaixo de 20% é crítica; abaixo de 50% é baixa. Se houver excedente e a reserva já estiver em 100%, o programa sugere ligar sistemas não essenciais ou a estufa. Ele não calcula quanto foi carregado ou retirado da bateria, nem muda a porcentagem da reserva após o saldo.

5.2 Emergência e tempestade de poeira

Em emergência, as prioridades 2 e 3 são desligadas. Restam oxigênio, aquecimento e água, que somam **35 kWh**. Se a geração prevista ainda for menor que esse valor, a saída mostra o déficit dos essenciais. Mantê-los ligados na árvore não garante que a energia disponível seja suficiente.

Com poeira, fazer_previsoes considera apenas **30% da previsão solar**. No mesmo cenário, a solar cai de 22,70 para 6,81 kWh e a geração total cai para 34,46 kWh. O fator de 30% é uma simplificação do programa, e não um cálculo físico do efeito de cada tempestade.

6. Organização do código e execução

O main.py coordena as etapas mostradas na Figura 3. Primeiro ele lê o consumo da árvore; depois calcula as previsões, compara a geração com o consumo e aplica as regras. Quando há cortes, soma o consumo novamente e refaz a análise. Para executar, usamos **python main.py** ou **py -3 main.py** no Windows.

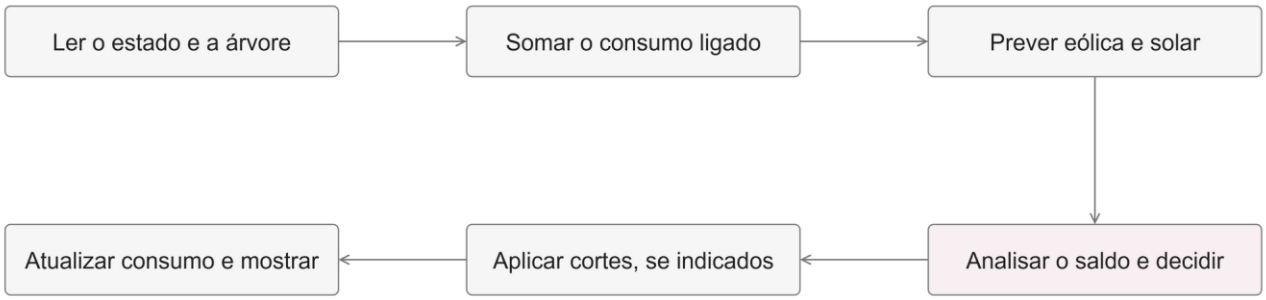


Figura 3 - Etapas executadas pelo sistema da equipe

Arquivo	O que faz
dados.py	Guarda o estado, os históricos e a árvore dos sistemas
estruturas.py	Busca, navega, soma o consumo e desliga por prioridade
previsao.py	Ajusta as retas, faz previsões e calcula a tendência do consumo
analise_energia.py	Calcula o saldo, os avisos e a cobertura do consumo
regras.py	Escolhe o modo e os equipamentos que podem ser desligados
main.py	Junta as etapas e mostra a demonstração no terminal

6.1 Casos conferidos no código

Caso	Resultado observado
Cenário inicial: reserva 40% e consumo 70 kWh	ECONOMIA; consumo cai para 45 kWh
Reserva 15%	EMERGENCIA; corte das prioridades 2 e 3
Reserva 40% e temperatura -81 °C	EMERGENCIA por frio extremo com reserva baixa
Reserva 80%, consumo 70 e geração 50,35 kWh	ATENCAO e aviso preventivo; sem corte automático
Reserva 45% e consumo 50 kWh	NORMAL com alerta para reduzir consumo; sem corte
Poeira com os demais dados iniciais	Solar 6,81 kWh; geração total 34,46 kWh

Também foram conferidos os limites de reserva de 20% e 50%, o consumo de 60 kWh e a temperatura limite de frio extremo. As comparações do código são estritas: usam menor que ou maior que. O ajuste da reta rejeitou listas de tamanhos diferentes e entradas históricas sem variação.

7. Conclusão e limites

A execução reúne os dados, prevê a geração e desliga primeiro os equipamentos menos importantes. No cenário padrão, essa decisão elimina o déficit previsto e deixa saldo de 5,35 kWh. A simulação usa apenas a biblioteca padrão do Python, executa um ciclo e não controla equipamentos reais. Ela não calcula a carga final da bateria nem garante as previsões para condições fora dos exemplos históricos.